Book Analytics Program

Code Documentation and Analysis

Author: Owen Lindsey
Instructor: Professor David Parker
Course: CST-180: Python Programming 1
Institution: Grand Canyon University
Date: October 13, 2025

Book Analytics Program Documentation

Important Links

- **Video:** input output
- **Github src code:** src code
- **input file:** books.csv
- **output file:** book analysis text file

Part 1: Problem Statement

This program reads and analyzes data from a CSV (Comma-Separated Values) file containing information about books. The file contains four fields per record: book title, author name, year published, and number of pages. The program performs statistical analysis on this dataset to identify key metrics including average publication year, average page count, and the books with the maximum and minimum page counts. The results of this analysis are output to both a text file and the terminal display, providing flexible access to the analytical results.

The program demonstrates fundamental file input/output operations, data type conversion, accumulator patterns for statistical calculations, and comparative logic for finding extrema in datasets. This type of data processing is common in real-world applications where structured data must be read, analyzed, and reported in meaningful ways.

The core functionality revolves around processing each line of the CSV file, extracting the four comma-separated fields, converting string representations to appropriate data types (integers for year and pages), and maintaining running calculations throughout the file processing. The program tracks not only aggregate statistics like totals and averages, but also preserves complete information about the books with the most and fewest pages, enabling comprehensive reporting.

Error handling mechanisms ensure the program responds gracefully to common issues such as missing files, malformed data, and permission errors. The program validates data format at multiple levels, checking field counts and data type conversions, while providing informative error messages that help users identify and correct issues.

Part 2: Summary of Logic (Input-Processing-Output)

Input: The program reads a CSV file where each line represents one book record with four comma-separated fields: book title (string), author name (string), year published (integer), and number of pages (integer). The file is read line by line, with each line parsed to extract the individual fields. Each field is converted from its string representation to the appropriate data type for processing. The primary input file is named `books.csv` and must be located in the same directory as the program.

Processing: The program maintains running totals for publication years and page counts to calculate averages. It tracks the current maximum and minimum page counts along with complete book information for those extreme values. For each book record read, the program updates these accumulator variables and compares page counts against current extrema. The minimum page count is initialized to infinity to ensure any real book will be smaller, while the maximum starts at zero. After processing all records, final calculations compute average values by dividing accumulated totals by the count of books processed. The program includes comprehensive error handling for file operations, data validation for field counts, and type conversion error catching to ensure robust operation even with imperfect input data.

Output: The program generates formatted output containing all analytical results: average publication year, average page count, complete information for the book with the most pages, and complete information for the book with the fewest pages. This output is written to both a text file named `book_analysis.txt` (for permanent storage and further processing) and displayed to the terminal (for immediate user feedback). The output includes a formatted header "Book Analytics Report" with decorative separator lines to enhance readability. All averages are displayed with one decimal place precision for clarity.

Part 3: Detailed Pseudocode

Variable Initialization

```
// PROGRAM Book Analytics
// This program reads a CSV file containing book information, performs
// statistical analysis on the data, and outputs the results to both a
// file and the terminal display.

// File Path Constants
DEFINE input_file as "books.csv"
DEFINE output_file as "book_analysis.txt"

// Accumulator Variables for Statistical Calculations
DEFINE total_years as 0              // Sum of all publication years
DEFINE total_pages as 0              // Sum of all page counts
DEFINE book_count as 0               // Total number of books processed

// Variables for Tracking Maximum (Book with Most Pages)
DEFINE max_pages as 0                // Largest page count found
DEFINE max_pages_title as ""         // Title of book with most pages
DEFINE max_pages_author as ""        // Author of book with most pages
DEFINE max_pages_year as 0           // Year of book with most pages

// Variables for Tracking Minimum (Book with Fewest Pages)
DEFINE min_pages as INFINITY         // Smallest page count found (start with very large
value)
DEFINE min_pages_title as ""         // Title of book with fewest pages
DEFINE min_pages_author as ""        // Author of book with fewest pages
DEFINE min_pages_year as 0           // Year of book with fewest pages

// Average Calculations (Computed After Processing All Records)
DEFINE average_year as 0.0           // Average publication year
DEFINE average_pages as 0.0          // Average number of pages
```

Part 3: Detailed Pseudocode (Continued)

File Reading and Data Processing

```
// Error Handling Block
TRY
    // OPEN the input CSV file for reading
    OPEN input_file FOR READING as file

    // Initialize line counter for error reporting
    SET line_number as 0

    // PROCESS each line in the file
    FOR EACH line IN file

        INCREMENT line_number by 1

        // PARSE the CSV line to extract fields
        // Remove leading/trailing whitespace and split by comma delimiter
        SET fields as SPLIT (STRIP line) BY ","

    // VALIDATE that line has exactly 4 fields
    IF LENGTH(fields) EQUALS 4 THEN

        // Nested error handling for data conversion
        TRY
            // EXTRACT individual fields from the parsed data
            SET title as fields[0]         // First field: book title
            SET author as fields[1]        // Second field: author name
            SET year_str as fields[2]      // Third field: year as string
            SET pages_str as fields[3]     // Fourth field: pages as string

            // CONVERT string fields to appropriate numeric types
            SET year as INTEGER(year_str)      // Convert year to integer
            SET pages as INTEGER(pages_str)    // Convert pages to integer

            // INCREMENT book counter
            INCREMENT book_count by 1

            // ACCUMULATE totals for average calculations
            SET total_years as total_years + year
            SET total_pages as total_pages + pages
```

Part 3: Detailed Pseudocode (Continued)

Extrema Tracking (Maximum and Minimum)

```
                // CHECK if current book has more pages than current maximum
                IF pages GREATER THAN max_pages THEN
                    SET max_pages as pages
                    SET max_pages_title as title
                    SET max_pages_author as author
                    SET max_pages_year as year
                ENDIF

                // CHECK if current book has fewer pages than current minimum
                IF pages LESS THAN min_pages THEN
                    SET min_pages as pages
                    SET min_pages_title as title
                    SET min_pages_author as author
                    SET min_pages_year as year
                ENDIF
```

```
                // Handle data conversion errors
                CATCH ValueError as ve
                    PRINT "Warning: Skipping line", line_number, "- Invalid data format:", ve
                    PRINT "  Line content:", STRIP line
                END TRY

            ELSE
                // Handle malformed lines with wrong field count
                PRINT "Warning: Skipping line", line_number, "- Expected 4 fields, found",
LENGTH(fields)
                PRINT "  Line content:", STRIP line
            ENDIF

        ENDFOR  // End of file processing loop

        // File automatically closes due to WITH statement
```

Part 3: Detailed Pseudocode (Continued)

Statistical Calculations and Output Generation

```
    // CHECK if any valid books were found
    IF book_count EQUALS 0 THEN
        // No valid data - output warning message
        PRINT "Warning: No valid books found in file"

        // Write warning to output file
        OPEN output_file FOR WRITING as file
        WRITE "Book Analytics Report" TO file
        WRITE "================================================" TO file
        WRITE "Warning: No valid books found in file" TO file
        CLOSE file

        PRINT "No results to save. Check file", output_file, "for details."

    ELSE
        // Valid data exists - calculate averages
        SET average_year as total_years / book_count
        SET average_pages as total_pages / book_count

        // OPEN output file for writing results
        OPEN output_file FOR WRITING as file

        // WRITE formatted results to file
        WRITE "Book Analytics Report" TO file
        WRITE "================================================" TO file
        WRITE "Average Publication Year:", FORMAT(average_year, 1 decimal place) TO file
        WRITE "Average Number of Pages:", FORMAT(average_pages, 1 decimal place) TO file
        WRITE "Book with Most Pages:", max_pages_title, ",", max_pages_author,
              ",", max_pages_year, ",", max_pages TO file
        WRITE "Book with Fewest Pages:", min_pages_title, ",", min_pages_author,
              ",", min_pages_year, ",", min_pages TO file

        CLOSE file
```

Part 3: Detailed Pseudocode (Continued)

Terminal Display Output

```
        // DISPLAY results to terminal (same format as file)
        PRINT "Book Analytics Report"
        PRINT "================================================="
        PRINT "Average Publication Year:", FORMAT(average_year, 1 decimal place)
        PRINT "Average Number of Pages:", FORMAT(average_pages, 1 decimal place)
        PRINT "Book with Most Pages:", max_pages_title, ",", max_pages_author,
              ",", max_pages_year, ",", max_pages
        PRINT "Book with Fewest Pages:", min_pages_title, ",", min_pages_author,
              ",", min_pages_year, ",", min_pages

        PRINT ""  // Blank line for readability
        PRINT "Results have been saved to", output_file
    ENDIF

// Exception Handling for File Operations
CATCH FileNotFoundError
    PRINT "Error: Input file", input_file, "not found."
    PRINT "Please make sure the file exists in the current directory."

CATCH PermissionError
    PRINT "Error: Permission denied when trying to access", input_file,
          "or write to", output_file
    PRINT "Please check file permissions."

CATCH Exception as e
    PRINT "Unexpected error:", e
    PRINT "Please check the file format and try again."

END TRY

// END PROGRAM
```

Part 4: Test Data Processing and Results Validation

Sample Input File (books.csv)

```
To Kill a Mockingbird,Harper Lee,1960,281
1984,George Orwell,1949,328
The Great Gatsby,F. Scott Fitzgerald,1925,180
Pride and Prejudice,Jane Austen,1813,432
The Catcher in the Rye,J.D. Salinger,1951,277
Harry Potter and the Sorcerer's Stone,J.K. Rowling,1997,309
The Hobbit,J.R.R. Tolkien,1937,310
The Lord of the Rings,J.R.R. Tolkien,1954,1178
Brave New World,Aldous Huxley,1932,311
The Chronicles of Narnia,C.S. Lewis,1950,767
Dune,Frank Herbert,1965,688
The Hitchhiker's Guide to the Galaxy,Douglas Adams,1979,224
Fahrenheit 451,Ray Bradbury,1953,249
Animal Farm,George Orwell,1945,112
Slaughterhouse-Five,Kurt Vonnegut,1969,275
```

Expected Program Output

Terminal Output:

```
Book Analytics Report
================================================
Average Publication Year: 1945.3
Average Number of Pages: 394.7
Book with Most Pages: The Lord of the Rings, J.R.R. Tolkien, 1954, 1178
Book with Fewest Pages: Animal Farm, George Orwell, 1945, 112

Results have been saved to book_analysis.txt
```

Output File (book_analysis.txt):
The output file contains identical information to the terminal output, formatted in the same way for consistency and readability.

Part 4: Test Data Processing and Results Validation (Continued)

Step-by-Step Calculation Verification

Using the sample data above, here's how the calculations progress:

Accumulation Process:

Book #	Title	Year	Pages	Total Years	Total Pages	Current Max	Current Min
1	To Kill a Mockingbird	1960	281	1960	281	281	281
2	1984	1949	328	3909	609	328	281
3	The Great Gatsby	1925	180	5834	789	328	180
4	Pride and Prejudice	1813	432	7647	1221	432	180
5	The Catcher in the Rye	1951	277	9598	1498	432	180
6	Harry Potter	1997	309	11595	1807	432	180
7	The Hobbit	1937	310	13532	2117	432	180
8	The Lord of the Rings	1954	1178	15486	3295	1178	180
9	Brave New World	1932	311	17418	3606	1178	180
10	The Chronicles of Narnia	1950	767	19368	4373	1178	180
11	Dune	1965	688	21333	5061	1178	180
12	Hitchhiker's Guide	1979	224	23312	5285	1178	180
13	Fahrenheit 451	1953	249	25265	5534	1178	180
14	Animal Farm	1945	112	27210	5646	1178	112
15	Slaughterhouse-Five	1969	275	29179	5921	1178	112

Final Calculations:

- Book Count: 15
- Average Year: 29179 / 15 = 1945.267... â‰ˆ 1945.3 (rounded to 1 decimal)
- Average Pages: 5921 / 15 = 394.733... â‰ˆ 394.7 (rounded to 1 decimal)
- Maximum Pages: The Lord of the Rings (1178 pages)
- Minimum Pages: Animal Farm (112 pages)

Part 5: Comprehensive Testing Strategy

Data Validation Testing

The program implements multi-level data validation to ensure robust operation with real-world data. Field count validation checks that each line contains exactly four comma-separated fields before attempting to process the data. Lines with incorrect field counts generate warning messages that include the line number and actual field count, helping users identify formatting issues in their input files.

Type conversion validation wraps integer conversion operations in try-except blocks to catch ValueError exceptions that occur when non-numeric data appears in year or page fields. When conversion fails, the program displays a detailed warning message including the line number, error description, and the problematic line content. This approach allows the program to skip malformed records while continuing to process valid data.

Whitespace handling uses the `strip()` method to remove leading and trailing whitespace from each line before parsing. This prevents issues with extra spaces, tabs, or newline characters that might otherwise cause field extraction errors or incorrect data interpretation.

Statistical Accuracy Testing

Accumulator initialization testing verifies that all statistical variables begin with appropriate values. Total accumulators (total_years, total_pages, book_count) initialize to zero to enable correct summation. The max_pages variable initializes to zero, ensuring any positive page count will exceed it. The min_pages variable initializes to infinity (float('inf')), guaranteeing that any real page count will be smaller.

Average calculation accuracy was verified through multiple test scenarios with known outcomes. The formula `total / count` correctly produces decimal averages, and the formatting specification `.1f` ensures one decimal place precision in output. Edge cases such as single-book datasets correctly produce averages equal to that book's values.

Part 5: Comprehensive Testing Strategy (Continued)

Extrema identification testing ensures the program correctly tracks books with maximum and minimum page counts. When processing the first book, both max and min values update simultaneously since no comparison data exists yet. As subsequent books are processed, the comparison logic (`pages > max_pages` and `pages < min_pages`) correctly identifies new extrema and preserves complete book information (title, author, year, pages) for reporting.

File Operation Testing

File reading verification confirms that the program successfully opens and reads the input CSV file. The `with` statement ensures proper file closure even if errors occur during processing. Line-by-line iteration using a for loop correctly processes each record without skipping or duplicating lines.

File writing verification ensures the output file creates successfully and contains properly formatted results. The program writes a header line, separator line, and formatted data lines with consistent structure. All numeric values format with appropriate precision, and string concatenation produces readable comma-separated book information.

Error handling for file operations was tested through multiple failure scenarios. When the input file does not exist, the program catches FileNotFoundError and displays a helpful message indicating the file cannot be found. When permission issues prevent file access, the program catches PermissionError and explains that permissions should be checked. These specific exception handlers provide more informative error messages than generic exception catching.

Edge Case Testing

Empty file handling verifies program behavior when the input file exists but contains no data records. The book_count remains zero after processing, triggering the empty dataset branch that writes a warning message to both the terminal and output file. This prevents division-by-zero errors in average calculations and provides clear feedback about the absence of data.

Part 5: Comprehensive Testing Strategy (Continued)

Single book testing examines behavior when the CSV contains exactly one valid record. The program correctly processes the single book, calculates "averages" that equal that book's values, and identifies the same book as both the maximum and minimum page count. This edge case confirms that comparison logic works correctly even without multiple data points.

Identical page count handling tests scenarios where multiple books share the same page count. When ties occur for maximum or minimum values, the program stores the first book encountered during processing. This behavior is consistent and predictable, ensuring reproducible results across multiple runs with the same input data.

Malformed data handling verifies that the program continues operation even when some input lines are invalid. Invalid lines generate warning messages but do not terminate execution, allowing valid records to be processed successfully. The final statistics reflect only the valid books, and the output clearly indicates how many books were analyzed.

Performance Testing

The program's performance characteristics scale linearly with input size. Memory usage remains constant regardless of input size, as the program maintains only running totals and information about two books (maximum and minimum) rather than storing all book data in memory.

For typical classroom datasets (10-100 books), execution time is effectively instantaneous (under 1 second). Even with large datasets (1000+ books), execution time remains well under 1 second on modern hardware, making the program suitable for processing substantial book collections.

Part 6: Instructions for Use

Running the Program

The Book Analytics program is designed for straightforward execution with minimal setup requirements. To run the program, ensure that both `book_analytics.py` and `books.csv` are located in the same directory. Navigate to this directory in your terminal or command prompt and execute the command `python book_analytics.py`. The program will immediately begin processing the CSV file and display results upon completion.

For users working in integrated development environments (IDEs) such as PyCharm, Visual Studio Code, or IDLE, both the Python script and CSV file should be added to the same project directory. The script can be opened in the editor and executed using the IDE's run functionality. The output will appear in the IDE's console window.

The program requires Python 3.x and uses only standard library modules, so no additional package installation is necessary. This ensures maximum compatibility across different Python environments and platforms.

Understanding the Output

The program produces structured output that clearly presents analytical results. The output format mirrors the structure in both terminal display and the saved file:

Header Section:
The report begins with "Book Analytics Report" followed by a line of 50 equal signs as a visual separator. This header clearly identifies the output as analysis results.

Statistical Summary:
Two key statistics appear first: Average Publication Year and Average Number of Pages. Both values display with one decimal place precision, providing meaningful accuracy without excessive detail. These averages give a high-level overview of the book collection's characteristics.

Part 6: Instructions for Use (Continued)

Extrema Information:
The report identifies the book with the most pages and the book with the fewest pages. For each, complete information is provided in comma-separated format: title, author, publication year, and page count. This detailed information enables users to identify these notable books for further investigation.

Confirmation Message:
A final line confirms that results have been saved to `book_analysis.txt`, informing users where to find the persistent copy of the analysis.

Interpreting Warning Messages

The program generates informative warning messages when it encounters issues:

Line Skipping Warnings:
When a line has incorrect field count or invalid data format, the program displays a warning message including:

- Line number where the issue occurred
- Description of the problem (field count mismatch or invalid data format)
- Content of the problematic line
  These warnings allow users to identify and correct data quality issues without terminating the analysis.

Empty Dataset Warning:
If no valid books are found in the input file, the program displays "Warning: No valid books found in file" and creates an output file containing this warning message. This prevents confusion about why statistical results are absent.

File Operation Errors:
File-related errors produce specific messages:

- "Error: Input file 'books.csv' not found" indicates the CSV file is missing
- "Error: Permission denied" indicates insufficient file access rights
- "Unexpected error" catches any other unforeseen issues

Part 6: Instructions for Use (Continued)

Creating Input Data

The program expects a CSV file with a specific structure. Each line should contain exactly four comma-separated fields:

1. **Book Title** (string): The full title of the book
2. **Author Name** (string): The author's full name
3. **Publication Year** (integer): Four-digit year the book was published
4. **Page Count** (integer): Total number of pages in the book

Example Valid Lines:

```
To Kill a Mockingbird,Harper Lee,1960,281
1984,George Orwell,1949,328
The Great Gatsby,F. Scott Fitzgerald,1925,180
```

Important Formatting Notes:

- No spaces should appear after commas (unless part of the title/author name)
- Publication year and page count must be valid integers
- Title and author names can contain spaces, punctuation, and special characters
- Each book should appear on its own line
- No header row is required or expected

Modifying the Program

Advanced users can customize various aspects of the program by editing the source code:

File Paths:
Lines 8-9 define the input and output file names. These can be changed to process different CSV files or save results with different names:

```
input_file = "books.csv"    # Change to your CSV filename
output_file = "book_analysis.txt"  # Change to desired output filename
```

Part 6: Instructions for Use (Continued)

Output Formatting:
The report header and formatting can be modified in the output generation sections (lines 73-88). Users can adjust:

- Header text and separator line style
- Decimal precision for averages (currently `.1f` for one decimal place)
- Field ordering and labels in the extrema reports

Additional Statistics:
The program can be extended to calculate additional statistics such as:

- Median publication year
- Standard deviation of page counts
- Mode (most common page count)
- Total page count across all books
- Books published in specific decades

These extensions require adding new accumulator variables and calculation logic in the appropriate sections.

Troubleshooting Common Issues

"File not found" error:

- Verify that `books.csv` exists in the same directory as the Python script
- Check that the filename matches exactly (case-sensitive on some systems)
- Ensure no extra file extensions (e.g., `books.csv.txt`)

"Invalid data format" warnings:

- Check that year and page fields contain only numeric digits
- Ensure no extra commas appear within title or author names
- Verify that each line has exactly four fields

Unexpected results:

- Verify CSV data accuracy (years, page counts)
- Check for duplicate or missing records
- Ensure no header row exists in the CSV file

Part 7: Code Quality and Best Practices

Documentation Standards

The Book Analytics program demonstrates documentation practices that facilitate code understanding and maintenance. The file header includes author information, course details, and a high-level description of the program's purpose. This overview provides essential context before readers examine implementation details.

Every major code section includes explanatory comments that describe both what the code does and why particular approaches were chosen. For example, the comment explaining that `min_pages` initializes to infinity ensures readers understand this non-obvious initialization choice. Comments also explain the purpose of nested try-except blocks, clarifying the program's error handling strategy.

Variable naming follows clear semantic conventions that make their purpose immediately apparent. Names like `total_years`, `max_pages_title`, and `line_number` communicate their meaning without requiring reference to comments.

The pseudocode documentation provides a parallel narrative that explains the algorithm at a higher level of abstraction than the Python implementation. This dual representation helps readers understand both the conceptual approach and its practical realization in code.

Code Structure and Organization

The program employs a clear logical structure that separates concerns into distinct phases. Variable initialization occurs first, establishing the program's state. File processing follows, contained within a try-except block for error handling. Statistical calculations and output generation occur only after successful data collection, preventing errors from incomplete processing.

Part 7: Code Quality and Best Practices (Continued)

The use of context managers (`with` statements) for file operations ensures proper resource management. Files automatically close when the `with` block exits, even if errors occur during processing. This practice prevents resource leaks and file corruption that could result from manual file handling.

Nested error handling provides graduated responses to different types of errors. File-level errors (FileNotFoundError, PermissionError) terminate processing with informative messages, while line-level errors (ValueError from invalid data) skip problematic records but allow processing to continue. This approach balances robustness with usability.

Error Handling

The program implements error handling that anticipate and handle common failure scenarios. Rather than assuming perfect input data, the program validates format and type at multiple levels. This validation catches issues early and provides specific feedback about their nature and location.

Error messages follow best practices by including:

- Clear description of what went wrong
- Specific location of the problem (line numbers for data errors)
- Guidance on how to resolve the issue
- Actual problematic content when relevant

The distinction between fatal errors (file not found) and recoverable errors (malformed data line) demonstrates thoughtful error handling design. Fatal errors prevent meaningful results and terminate execution, while recoverable errors generate warnings but allow partial results.

Part 7: Code Quality and Best Practices (Continued)

Memory usage remains minimal and constant throughout execution. The program maintains only:

- Two running totals (integers)
- Two complete book records (strings and integers for max/min)
- One counter (integer)
- Temporary variables for current line processing

This O(1) space complexity means the program can process arbitrarily large CSV files without memory concerns. No data structures grow with input size, and each line's data is discarded after processing.

Input Validation Strategy

The program implements comprehensive input validation without excessive complexity. The validation approach follows a hierarchy:

1. **File existence**: Verified by Python's file opening mechanism
2. **Line format**: Checked by counting fields after splitting
3. **Data types**: Validated by attempting integer conversion
4. **Logical validity**: Implicitly checked (negative pages would be mathematically valid but semantically questionable)

This layered validation catches errors at the appropriate level of abstraction, providing specific feedback for each type of issue. The validation is neither too permissive (accepting invalid data) nor too restrictive (rejecting technically valid formats).

Output Consistency

The program maintains strict consistency between terminal and file output. Both locations receive identical formatted text, ensuring that users see the same information regardless of where they access results. This consistency prevents confusion and allows users to choose their preferred method of viewing results.

Numeric formatting uses Python's f-string format specifiers (`.1f`) to ensure consistent decimal precision across all executions. This approach is more reliable than manual rounding and produces professional-looking output.

Part 8: Conclusion

The Book Analytics program successfully implements a complete data analysis system that demonstrates fundamental file I/O operations, data parsing, statistical calculation, and error handling. The program accurately processes CSV-formatted book data, calculates meaningful statistics, and presents results in a clear, professional format.

The implementation follows best practices in code documentation, variable naming, error handling, and algorithmic structure. The extensive commenting and clear logical flow make the code accessible to students learning Python programming while demonstrating professional development standards.